# BLOCKCHAIN-BASED LAND REGISTRY SYSTEM

**A PROJECT REPORT**

*Submitted by*

**Team BlockCodeM**

**Aditya Dhabhai [RA2512055010013]**

**Ishan Roy Barman [RA2512055010018]**

*Under the Guidance of*

**Kavi Priya D**

Assistant Professor, Department of Computing Technologies

*in partial fulfilment of the requirements for the course*

**Cloud Computing and Blockchain**

**M.Tech Computer Science and Engineering**




**DEPARTMENT OF COMPUTING TECHNOLOGIES**

**COLLEGE OF ENGINEERING AND TECHNOLOGY**

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**

**KATTANKULATHUR – 603203**

**NOVEMBER 2026**

# SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
# KATTANKULATHUR – 603203

## BONAFIDE CERTIFICATE

Certified that this Cloud Computing and Blockchain mini project report titled "**BLOCKCHAIN-BASED LAND REGISTRY SYSTEM**" is the bonafide work of "**Aditya Dhabhai [RA2512055010013]** and **Ishan Roy Barman [RA2512055010018]**" (Team BlockCodeM) who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

**FACULTY**
**SIGNATURE**
Kavi Priya D
Assistant Professor
Department of Computing Technologies
SRM Institute of Science and Technology,
Kattankulathur - 603203

**HOD**
**SIGNATURE**
Dr Niranjana G
Head of Department
Department of Computing Technologies
SRM Institute of Science and Technology,
Kattankulathur - 603203

# TABLE OF CONTENTS


| Chapter | Chapter Title | Sub-sections | Page No. |
|---|---|---|---|
| 1 | Introduction | 1.1 Overview of the Project | 1 |
| | | 1.2 Objectives of the Project | 1 |
| | | 1.3 Problem Definition | 2 |
| | | 1.4 Scope of the Application | 2 |
| | | 1.5 Methodology | 2 |
| | | 1.6 Technologies and Tools Used | 3 |
| 2 | Literature Survey and System Analysis | 2.1 Existing System | 4 |
| | | 2.2 Literature Survey | 4 |
| | | 2.3 Limitations of the Existing System | 4 |
| | | 2.4 Proposed System | 5 |
| | | 2.5 Feasibility Study | 5 |
| 3 | System Design | 3.1 System Architecture | 6 |
| | | 3.2 Module Description | 6 |
| | | 3.3 Data Flow Design | 7 |
| | | 3.4 Smart Contract Design | 7 |
| | | 3.5 Off-chain Data and Storage Design | 8 |
| | | 3.6 User Interface Design | 8 |
| 4 | Implementation | 4.1 Smart Contract Implementation | 9 |
| | | 4.2 Contract Compilation and Deployment | 9 |
| | | 4.3 Backend REST API | 10 |
| | | 4.4 Blockchain Connectivity | 11 |
| | | 4.5 Cloud Storage Integration | 11 |
| | | 4.6 Authentication and Middleware | 11 |
| | | 4.7 Frontend Implementation | 12 |
| 5 | Cloud Deployment and MetaMask Integration | 5.1 AWS Deployment Architecture | 13 |
| | | 5.2 Storage and Database on AWS | 13 |
| | | 5.3 Compute, IAM and HTTPS | 14 |
| | | 5.4 MetaMask Integration | 16 |
| 6 | Testing and Validation | 6.1 Testing Objectives | 18 |
| | | 6.2 Types of Testing | 18 |
| | | 6.3 Test Cases | 18 |
| | | 6.4 Validation of Inputs | 19 |
| | | 6.5 Issues Found and Resolved | 19 |
| 7 | Output and Results | 7.1 Application Screenshots | 20 |
| | | 7.2 Output Explanation | 22 |
| | | 7.3 Performance Evaluation | 22 |
| 8 | Conclusion and Future Enhancements | 8.1 Summary | 24 |
| | | 8.2 Key Achievements | 24 |
| | | 8.3 Limitations | 24 |
| | | 8.4 Future Enhancements | 24 |
| | References | | 26 |

# LIST OF FIGURES


| Figure No. | Figure Title | Page No. |
|---|---|---|
| 3.1 | System architecture of the Blockchain Land Registry | 6 |
| 3.2 | Data flow for land registration / transfer | 7 |
| 3.3 | Document verification (tamper detection) flow | 7 |
| 3.4 | Smart contract data model (LandRegistry.sol) | 8 |
| 4.1 | Local Ethereum node log: deployment, registration and transfer transactions | 10 |
| 5.1 | Implemented AWS deployment architecture | 13 |
| 5.2 | Amazon S3: document bucket and website bucket | 14 |
| 5.3 | Amazon DynamoDB: Users and LandDocuments tables | 14 |
| 5.4 | IAM role LandRegistryEC2Role attached to the server | 15 |
| 5.5 | Amazon EC2 instance land-registry-server running | 15 |
| 5.6 | Amazon CloudFront distribution and cache behaviour for the API | 15 |
| 5.7 | Application served from the S3 website, with deed links from S3 | 16 |
| 5.8 | MetaMask transaction request for registering LAND202 | 17 |
| 5.9 | Land registered with a MetaMask signature (signer address shown) | 17 |
| 5.10 | Tampered deed detected directly on the blockchain through MetaMask | 17 |
| 7.1 | Home page (public Search / History tab) | 20 |
| 7.2 | Register New Land form with sale-deed PDF | 20 |
| 7.3 | Land registered on blockchain: document hash and transaction hash | 20 |
| 7.4 | Ownership transferred to Priya Sharma | 21 |
| 7.5 | Ownership timeline after transfer (2 records) | 21 |
| 7.6 | Genuine document: DOCUMENT VERIFIED | 21 |
| 7.7 | Edited document: TAMPERED / INVALID DOCUMENT | 22 |
| 7.8 | API latency and on-chain gas consumption | 23 |


# LIST OF TABLES


| Table No. | Table Title | Page No. |
|---|---|---|
| 1.1 | Technology stack | 3 |
| 2.1 | Summary of literature survey | 4 |
| 2.2 | Feasibility analysis | 5 |
| 3.1 | Module description | 6 |
| 3.2 | On-chain vs off-chain data placement | 8 |
| 4.1 | REST API endpoints | 10 |
| 5.1 | AWS resources used by the system | 13 |
| 6.1 | Functional and negative test cases | 18 |
| 6.2 | Issues found during testing and their fixes | 19 |
| 7.1 | API response time (n = 30 per operation) | 22 |
| 7.2 | Blockchain cost and accuracy metrics | 23 |

# ABSTRACT

Land records kept in centralised databases and paper registers can be altered by insiders, forged, or lost, which leads to fraud and long ownership disputes. This project, **"Blockchain-Based Land Registry System"**, developed by team BlockCodeM, records land ownership and every transfer on an Ethereum blockchain so that the history of a parcel becomes tamper-evident and publicly verifiable.

The system uses a hybrid on-chain / off-chain design. Each title document (PDF sale deed) is fingerprinted with SHA-256 and only the hash, together with the Land ID, owner, location and block timestamp, is stored in a **Solidity** smart contract deployed with **Hardhat** on a **Ganache** Ethereum network. The full document is stored in **Amazon S3** and registrar accounts and the document index in **Amazon DynamoDB**. The system is deployed on **AWS**: the API and blockchain node on **EC2** with an **IAM role**, the frontend on an **S3 static website**, and an **Amazon CloudFront** HTTPS endpoint. Registrars may sign transactions with their own **MetaMask** wallet. A **Node.js / Express** REST API signs blockchain transactions through **ethers.js**, protects write operations with **JWT** authentication and bcrypt-hashed passwords, and accepts only PDF uploads. A **React + Vite** frontend provides four functions: register land, transfer ownership, search the complete ownership timeline and verify a document.

Verification recomputes the hash of an uploaded file and compares it with the on-chain hash inside the contract. In testing, all 30 genuine documents were verified and all 30 edited documents were detected as tampered. Read operations completed in about 15 ms, write transactions in about 115 ms on the local chain, and a registration consumed about 0.37 million gas. The project shows that blockchain combined with cloud storage gives a low-cost, transparent and reliable land record system.

**Keywords:** Blockchain, Ethereum, Smart Contract, Solidity, Land Registry, SHA-256, AWS S3, DynamoDB, EC2, CloudFront, MetaMask, Tamper Detection.

# CHAPTER 1: INTRODUCTION

## 1.1 Overview of the Project

Land is one of the most valuable assets a citizen owns, yet the records that prove ownership are still largely maintained in centralised databases and paper registers at sub-registrar offices. Such records can be altered by an insider, lost in a disaster, or forged, and each of these leads to costly and long-running ownership disputes. The **Blockchain Land Registry** developed by team **BlockCodeM** is a web application that records land ownership and every subsequent transfer on an Ethereum blockchain, while the actual title documents are kept in cloud storage.

The system follows the industry-standard **hybrid on-chain / off-chain pattern**. Each uploaded sale deed (PDF) is fingerprinted with the SHA-256 algorithm; only this 64-character hash, together with the Land ID, owner name, location and a block timestamp, is written to the LandRegistry smart contract. The full document is stored in cloud object storage. Anyone can later upload a document and the system recomputes its hash and compares it with the hash on the chain, so a single changed byte in a deed is detected immediately.

The application has three layers: a **React + Vite** single-page frontend, a **Node.js / Express** REST backend that handles authentication, file upload, hashing and blockchain interaction through **ethers.js**, and a **Solidity** smart contract deployed on a local Ethereum network (Ganache / Hardhat Network). The complete system is deployed on **Amazon Web Services (AWS)**: deeds are stored in **Amazon S3**, registrar accounts and the document index in **Amazon DynamoDB**, the API and the blockchain node run on **Amazon EC2**, the React build is served from an **S3 static website**, and **Amazon CloudFront** provides the HTTPS endpoint. Registrars can also sign transactions with their own **MetaMask** wallet.

## 1.2 Objectives of the Project

1. **Tamper-proof ownership records:** store land records on an immutable blockchain ledger so that no single party can silently change them.
2. **Complete ownership history:** maintain an append-only timeline of every owner of a parcel with timestamps.
3. **Document authenticity verification:** allow any citizen, bank or court to check whether a deed is genuine by comparing its SHA-256 hash with the on-chain hash.
4. **Secure cloud storage of documents:** keep large PDF documents off-chain in cloud storage to save gas cost while keeping them retrievable.
5. **Controlled write access:** only authenticated registrars (JWT login) may register or transfer land; search and verification stay public.
6. **Cloud deployment:** run the complete system on AWS (S3, DynamoDB, EC2, IAM, CloudFront) using an IAM role instead of access keys.

7. **Wallet signing:** let registrars sign transactions with MetaMask so that the signer is recorded on-chain.

## 1.3 Problem Definition

Land records managed through centralised systems face the following problems:

- **Unauthorised modification:** a database administrator or a compromised account can edit ownership data without leaving a reliable trace.
- **Forged documents:** fake or edited sale deeds are hard to detect visually, which enables fraud such as selling the same land twice.
- **Opaque history:** previous owners and the chain of title are spread across physical registers and are slow to trace.
- **Single point of failure:** a central server or paper archive can be lost, damaged or made unavailable.
- **Slow verification:** banks and buyers must visit offices and wait days to confirm a title.

**Problem statement:** design and implement a blockchain-based system that records land ownership and transaction details securely, stores supporting documents off-chain in the cloud, and provides a transparent, verifiable and tamper-evident ownership history.

## 1.4 Scope of the Application

**Functional scope**

- Registrar sign-up and login with bcrypt-hashed passwords and 24-hour JWT tokens.
- Register a new land parcel (Land ID, owner, location and sale-deed PDF) with duplicate-ID protection.
- Transfer ownership with a new deed while the previous record stays in the history.
- Search any Land ID and view the complete ownership timeline from the blockchain.
- Verify any document against the current on-chain hash (VERIFIED / TAMPERED).
- Download the stored deeds of a parcel through time-limited S3 pre-signed links.
- Sign register / transfer with MetaMask, and verify a deed directly on the blockchain from the browser.

**Out of scope (current version)**

- Payment of stamp duty or sale consideration on-chain.
- Public mainnet / testnet deployment (the chain is a private Ganache network on EC2).
- Survey maps / GIS boundary data.

## 1.5 Methodology

The project followed an **iterative development model** aligned with the three course reviews:

| Phase | Activities | Review |
|---|---|---|
| 1. Problem identification | Study of land-record issues, literature survey of blockchain land registries | Review 1 (17 Aug 2026) |
| 2. Design | Architecture, smart contract data model, API design, UI wireframes | Review 2 (17 Sep 2026) |
| 3. Implementation | Solidity contract, Hardhat deployment, Express API, React UI, cloud upload, JWT auth | Review 2 / 3 |
| 4. Testing & evaluation | Functional and negative test cases, latency and gas measurements | Review 3 (27 Oct 2026) |
| 5. Cloud deployment & documentation | AWS deployment (S3, DynamoDB, EC2, CloudFront), MetaMask, final report | Final Review (11 Nov 2026) |

## 1.6 Technologies and Tools Used

**Table 1.1: Technology stack**

| Layer | Technology | Purpose |
|---|---|---|
| Smart contract | Solidity ^0.8.19 | LandRegistry contract (records, history, verification) |
| Blockchain network | Ganache / Hardhat Network | Local Ethereum chain for development and demo |
| Framework | Hardhat 2 + hardhat-toolbox | Compile and deploy contracts |
| Chain library | ethers.js v6 | JSON-RPC provider, wallet signing, contract calls |
| Backend | Node.js 22, Express 5 | REST API server |
| File upload | Multer 2 | Multipart parsing, PDF-only filter, 10 MB limit |
| Hashing | Node.js crypto (SHA-256) | Document fingerprint |
| Authentication | bcryptjs, jsonwebtoken | Password hashing, JWT tokens |
| Cloud storage | Amazon S3 (AWS SDK v3) | Private bucket for PDF deeds, pre-signed links |
| Database | Amazon DynamoDB | Users and LandDocuments tables (on-demand) |
| Hosting | Amazon EC2, S3 website, CloudFront | API + chain server, frontend, HTTPS endpoint |
| Access control | AWS IAM role | S3/DynamoDB access without access keys |
| Wallet | MetaMask + ethers BrowserProvider | User-signed transactions, on-chain verify |
| Frontend | React 19, Vite 8, Axios | Single-page user interface |
| Tools | VS Code, Git/GitHub, pm2, Playwright | Development, version control, process manager, UI testing |

# CHAPTER 2: LITERATURE SURVEY AND SYSTEM ANALYSIS

## 2.1 Existing System

In India, land records are maintained by state revenue departments and registered at sub-registrar offices. Programmes such as the **Digital India Land Records Modernization Programme (DILRMP)** and state portals such as Karnataka's **Bhoomi** have digitised records, but the data still lives in centralised government databases. Ownership transfer requires the physical presence of the parties, stamp-paper deeds and manual entry by officials. Verification by a buyer or a bank means requesting an encumbrance certificate and waiting for it to be issued.

## 2.2 Literature Survey

**Table 2.1: Summary of literature survey**

| # | Work | Key idea | Relevance to this project |
|---|---|---|---|
| 1 | Nakamoto (2008) [1] | Bitcoin: hash-linked blocks and proof-of-work give an append-only public ledger | Basis of immutability of our records |
| 2 | Buterin (2014) [2], Wood (2014) [3] | Ethereum adds a Turing-complete virtual machine and smart contracts | Our registry logic runs as a Solidity contract |
| 3 | Lantmäteriet testbed, Sweden (2017) [7] | National land authority pilot of property transfer on a blockchain | Shows real-world value of on-chain transfer history |
| 4 | Thakur et al. (2020) [4] | Framework and challenges for blockchain land titling in India | Indian context; motivates hash-on-chain design |
| 5 | Krishnapriya & Sarath (2020) [5] | Ethereum smart-contract based land registration | Similar register / transfer workflow |
| 6 | Shuaib et al. (2020) [6] | Secure and reliable blockchain land registry framework | Role separation and security considerations |
| 7 | Benet (2014) [9] | IPFS content-addressed off-chain storage | Supports off-chain document storage with hash links |

The survey shows that storing whole documents on a blockchain is expensive, and that the common practice is to keep a cryptographic hash on-chain and the file off-chain. It also shows that most academic prototypes stop at the smart contract and do not offer a complete web workflow with cloud storage and tamper verification. This project addresses that gap.

## 2.3 Limitations of the Existing System

- Records can be changed by privileged insiders without an independent audit trail.
- Forged or edited deeds cannot be detected without manual expert checks.
- History of ownership is fragmented and slow to obtain.
- Centralised storage is a single point of failure.

- Citizens cannot verify a record themselves in real time.

## 2.4 Proposed System

The proposed system stores each land record as a LandRecord structure in an Ethereum smart contract. Every registration or transfer is a signed transaction that is mined into a block, and the contract appends the new record to the parcel's history array, so earlier owners can never be removed. Supporting PDFs are uploaded to cloud storage and only their SHA-256 hashes are stored on-chain. A public verification endpoint lets anyone check a document in a few milliseconds.

**Features of the proposed system**

- Immutable, timestamped ownership timeline for every Land ID.
- Instant tamper detection by hash comparison inside the contract (verifyHash).
- Low on-chain cost: about 0.37 million gas per registration (Section 7.3).
- Role separation: registrars write (JWT protected), public users read and verify.
- Cloud document storage in Amazon S3 with time-limited pre-signed URLs.
- Optional MetaMask signing so each registrar's own address appears on-chain.

## 2.5 Feasibility Study

**Table 2.2: Feasibility analysis**

| Type | Assessment |
|---|---|
| Technical | All components are open-source and mature (Solidity, Hardhat, ethers.js, Express, React). The system runs on a normal laptop with Node.js 18+. |
| Economic | Development cost is zero: the private chain runs on one t3.micro EC2 instance and S3, DynamoDB and CloudFront stay within the AWS free plan. On a public chain, only small hashes are stored, which keeps gas cost low. |
| Operational | Registrars use a simple four-tab web UI; no blockchain knowledge is needed because the backend signs transactions by default, while MetaMask signing is optional. |
| Legal / social | Supports, rather than replaces, the legal registration process by adding an independent tamper-evident audit trail. |

# CHAPTER 3: SYSTEM DESIGN

## 3.1 System Architecture

Figure 3.1 shows the four-tier architecture. The React client talks only to the Express API. The API holds the server wallet key and reaches S3 and DynamoDB through the EC2 IAM role, so no secret is exposed to the browser. MetaMask talks to the chain only through the API's allow-listed /api/rpc gateway.

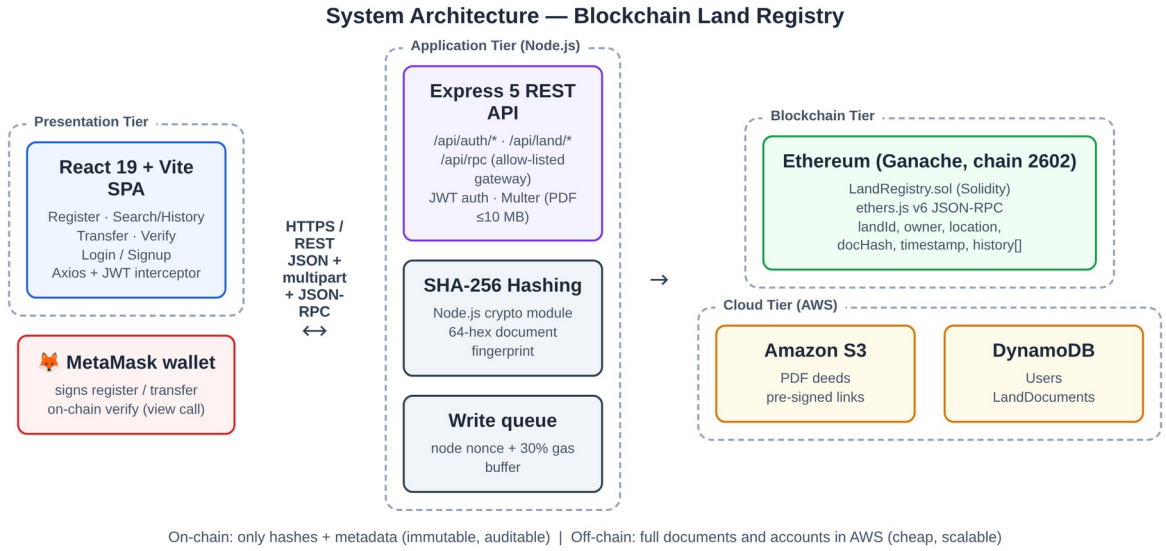



**Figure 3.1: System architecture of the Blockchain Land Registry**

## 3.2 Module Description

**Table 3.1: Module description**

| Module | File(s) | Responsibility |
|---|---|---|
| Smart contract | blockchain/contracts/LandRegistry.sol | Stores records and history, enforces no-duplicate and must-exist rules, verifies hashes |
| Deployment | blockchain/scripts/deploy.js, hardhat.config.cjs | Compiles and deploys contract to Ganache |
| Chain connector | backend/contract.js | ethers provider, wallet, contract; serialised write queue |
| REST API | backend/server.js | Land endpoints, hashing, RPC gateway, MetaMask prepare/record |
| Authentication | backend/auth.js | Signup / login, bcrypt, JWT issue and authMiddleware |
| Cloud storage | backend/storage.js | S3 upload and pre-signed download URLs |
| Database | backend/db.js | DynamoDB Users and LandDocuments access |
| Wallet | frontend/src/wallet.js, WalletBar.jsx | MetaMask connect, network add, signing, on-chain verify |
| User interface | frontend/src/App.jsx, components/*.jsx | Tabs: Register, Search/History, Transfer, Verify, Login, Signup |

| Module | File(s) | Responsibility |
|---|---|---|
| Deployment | deploy/ec2-setup.sh | One-command EC2 setup: Ganache + API under pm2 |

## 3.3 Data Flow Design

Figure 3.2 shows the step-by-step data flow when a registrar registers a parcel. Ownership transfer follows the same path but calls transferOwnership() and requires the Land ID to exist.

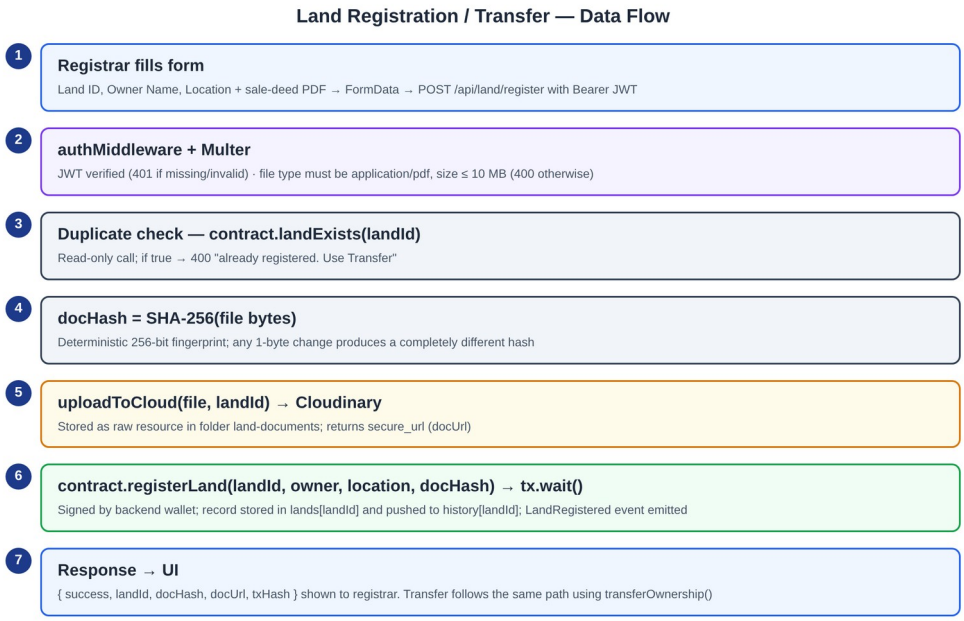



**Figure 3.2: Data flow for land registration / transfer**

Figure 3.3 shows the verification flow. Verification is a read-only (view) call, so it costs no gas and does not need a login.

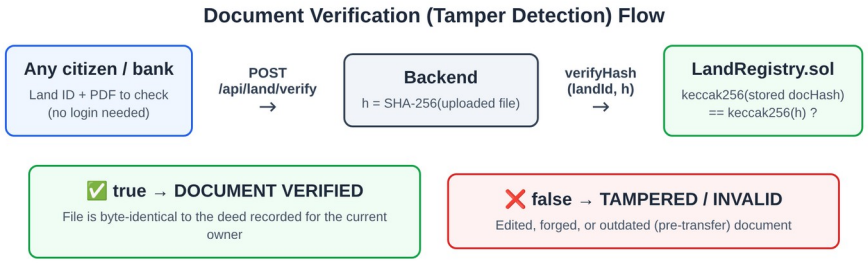



**Figure 3.3: Document verification (tamper detection) flow**

## 3.4 Smart Contract Design

The contract keeps two mappings keyed by Land ID: lands holds the current record for O(1) lookup, and history holds an append-only array of all records. Events are emitted for every write so that external tools can index the activity (Figure 3.4).

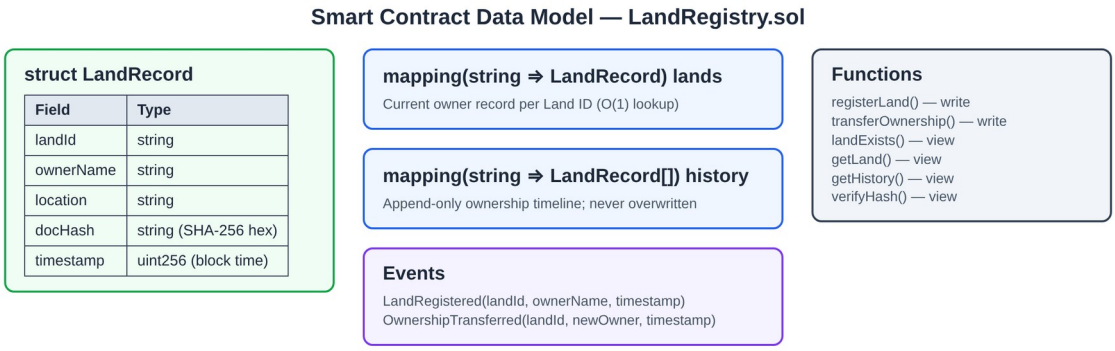



**Figure 3.4: Smart contract data model (LandRegistry.sol)**

## 3.5 Off-chain Data and Storage Design

**Table 3.2: On-chain vs off-chain data placement**

| Data | Where stored | Reason |
|---|---|---|
| Land ID, owner, location, docHash, timestamp | Ethereum contract storage | Must be immutable and publicly verifiable |
| Full PDF deed | Amazon S3 private bucket (land-documents/LANDID/time.pdf) | Large binary; too costly on-chain |
| Registrar accounts (name, email, bcrypt hash) | DynamoDB table Users (PK email) | Private data; must not be public on-chain |
| Document index (S3 key, hash, owner, tx, signer) | DynamoDB table LandDocuments (PK landId, SK uploadedAt) | Lists the deeds of a parcel for download |
| JWT secret, wallet key | backend/.env on EC2 (not committed); AWS access via IAM role | Secrets kept server-side only |

## 3.6 User Interface Design

The UI is a single page with a dark theme and four tabs. Register and Transfer show a lock icon until the registrar logs in; Search/History and Verify are always public. Each form shows a loading label while the blockchain transaction is being mined, and results (hash, transaction hash, timeline or verification verdict) are displayed in a highlighted card. Screenshots of every screen are given in Chapter 6.

# CHAPTER 4: IMPLEMENTATION

## 4.1 Smart Contract Implementation

The core write function registerLand() rejects duplicates, builds a LandRecord with the block timestamp, stores it and appends it to the history. transferOwnership() keeps the location, replaces the owner and document hash, and appends the new record.

```
function registerLand(string memory landId, string memory ownerName,
                      string memory location, string memory docHash) public {
    require(!landExists(landId), "Land already registered");
    LandRecord memory record = LandRecord({ landId: landId, ownerName:
ownerName,
        location: location, docHash: docHash, timestamp: block.timestamp });
    lands[landId] = record;
    history[landId].push(record);          // first entry of the timeline
    emit LandRegistered(landId, ownerName, block.timestamp);
}

function verifyHash(string memory landId, string memory hash) public view
returns (bool) {
    require(landExists(landId), "Land not registered");
    return keccak256(abi.encodePacked(lands[landId].docHash)) ==
           keccak256(abi.encodePacked(hash));
}
```

Solidity cannot compare strings directly, so verifyHash() compares the keccak256 digests of the stored and supplied hash strings.

## 4.2 Contract Compilation and Deployment

The contract is compiled with Hardhat and deployed with scripts/deploy.js (locally) or backend/scripts/deploy-local.js (on EC2, from the compiled bytecode). The deployed address is placed in backend/.env. Figure 4.1 shows the real node log from the demo run: the deployment transaction (1,425,762 gas), the registration of LAND101 and its transfer, each mined in its own block.

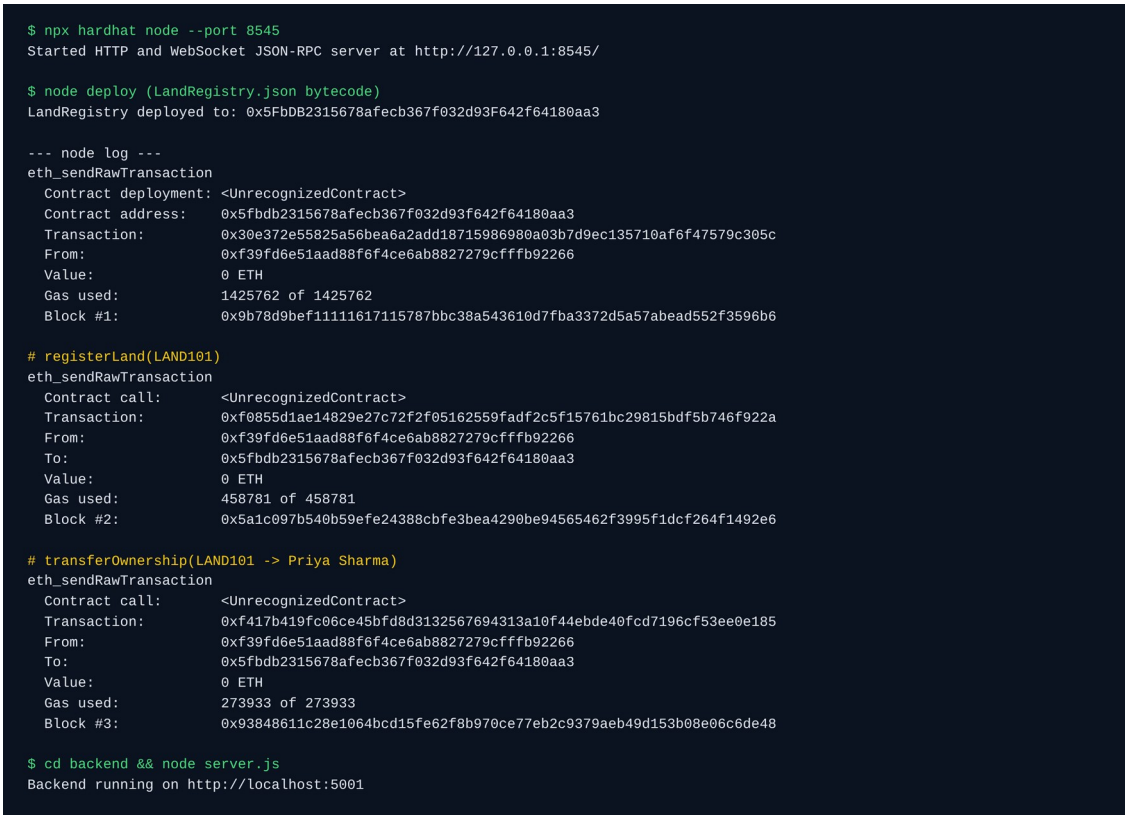



**Figure 4.1: Local Ethereum node log: deployment, registration and transfer transactions**

## 4.3 Backend REST API

**Table 4.1: REST API endpoints**

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| POST | /api/auth/signup | Public | Create registrar account (bcrypt hash) |
| POST | /api/auth/login | Public | Returns JWT (24 h) and user name |
| POST | /api/land/register | JWT | Multipart: landId, ownerName, location, document |
| POST | /api/land/transfer | JWT | Multipart: landId, newOwner, document |
| GET | /api/land/:id/history | Public | Full ownership timeline |
| POST | /api/land/verify | Public | Multipart: landId, document → verified true/false |
| GET | /api/land/:id/documents | Public | S3 pre-signed links of all deeds |
| GET / POST | /api/chain, /api/rpc | Public | Network details and allow-listed JSON-RPC for MetaMask |
| POST | /api/wallet/fund | JWT | 5 test ETH to the registrar's MetaMask wallet |
| POST | /api/land/prepare, /record | JWT | MetaMask flow: hash + S3 upload, then verify mined tx |

The document hash is computed on the server from the uploaded bytes, so a client cannot submit a fake hash:

```
function hashFile(path) {
  const fileBuffer = fs.readFileSync(path);
  return crypto.createHash("sha256").update(fileBuffer).digest("hex");
```

```
}
// inside POST /api/land/register
const docHash = hashFile(req.file.path);
const { key, url: docUrl } = await uploadDocument(req.file.path, landId); // S3
const tx = await sendWrite("registerLand", landId, ownerName, location,
docHash);
await recordDocument({ landId, s3Key: key, docHash, ownerName,
                       action: "register", txHash: tx.hash });     // DynamoDB
res.json({ success: true, landId, docHash, docUrl, txHash: tx.hash });
```

## 4.4 Blockchain Connectivity

backend/contract.js creates an ethers.js JsonRpcProvider for the Ganache URL, a Wallet from the server's private key and a Contract object from the compiled ABI. Read calls (landExists, getHistory, verifyHash) are free eth_call requests. Write calls go through a **serialised queue**: each transaction takes its nonce from the node and gets a 30% gas buffer over the estimate, which removed the "nonce too low" and out-of-gas errors seen during testing.

```
function sendWrite(method, ...args) {
  return enqueue(async (nonce) => {            // one write at a time
    const estimate = await contract[method].estimateGas(...args);
    return contract[method](...args, { nonce, gasLimit: (estimate * 13n) /
10n });
  });
}
```

## 4.5 Cloud Storage Integration

backend/storage.js uploads each PDF to the private S3 bucket under land-documents/LANDID/timestamp.pdf and returns a pre-signed download URL valid for one hour. backend/db.js stores registrar accounts in the DynamoDB Users table and one item per deed in LandDocuments, so the Search tab can list every deed of a parcel. The EC2 instance reaches both services through its IAM role, so no AWS access key exists in the code or the .env file. Without S3 settings the code falls back to Cloudinary and users.json for local development.

```
const key = `land-documents/${landId}/${Date.now()}.pdf`;
await s3.send(new PutObjectCommand({ Bucket: process.env.S3_BUCKET, Key: key,
  Body: fs.readFileSync(filePath), ContentType: "application/pdf" }));
const url = await getSignedUrl(s3,
  new GetObjectCommand({ Bucket: process.env.S3_BUCKET, Key: key }),
{ expiresIn: 3600 });
```

## 4.6 Authentication and Middleware

Passwords are hashed with bcrypt (10 salt rounds) and stored in DynamoDB with a conditional put, so the same e-mail cannot be registered twice. On login a JWT signed with JWT_SECRET

is issued for 24 hours. authMiddleware checks the Bearer token on register and transfer routes and returns 401 for a missing or invalid token. Multer accepts only application/pdf files up to 10 MB, a missing file returns 400, an unknown Land ID returns 404, and temporary upload files are deleted after every request.

## 4.7 Frontend Implementation

The React app (App.jsx) keeps the active tab, token and user name in state and in localStorage. A shared Axios instance (api.js) automatically adds the Authorization header to every request. Each feature is a separate component (RegistryLand, TransferLand, SearchLand, VerifyDoc, Login, Signup) that builds a FormData object, calls the API and renders the result or error.

# CHAPTER 5: CLOUD DEPLOYMENT AND METAMASK INTEGRATION

## 5.1 AWS Deployment Architecture

The complete application runs on Amazon Web Services in region ap-southeast-2 (Sydney), the region available on the account's free plan. Figure 5.1 shows the deployed services. The browser loads the React build from an S3 static website and calls the API through an Amazon CloudFront HTTPS endpoint; CloudFront forwards /api/* to the Express server on EC2. Ganache runs on the same instance on 127.0.0.1:8545, which is never opened to the internet.

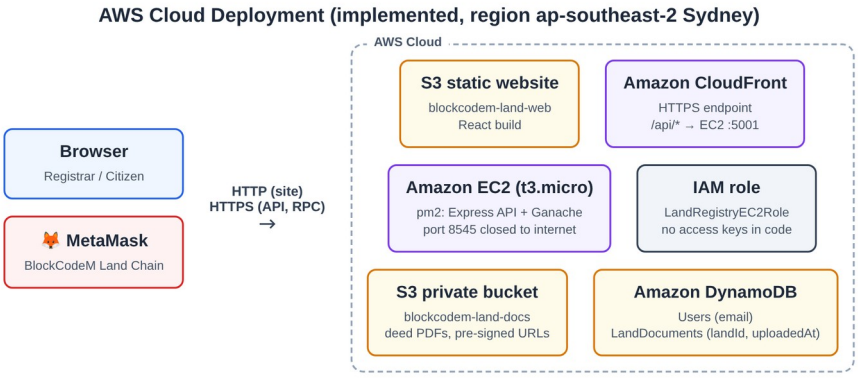



**Figure 5.1: Implemented AWS deployment architecture**

**Table 5.1: AWS resources used by the system**

| Service | Resource | Configuration and role |
|---|---|---|
| Amazon S3 | blockcodem-land-docs-125254 | Private bucket for PDF deeds; access only through 1-hour pre-signed URLs |
| Amazon S3 | blockcodem-land-web-125254 | Static website hosting of the React build (public read) |
| Amazon DynamoDB | Users, LandDocuments | On-demand tables: PK email; PK landId + SK uploadedAt |
| AWS IAM | LandRegistryEC2Role | Inline policy for S3 Put/Get and DynamoDB Get/Put/Query, plus SSM access |
| Amazon EC2 | land-registry-server (t3.micro) | Ubuntu 24.04; pm2 runs "chain" (Ganache, chain id 2602) and "api" (Express :5001) |
| Amazon CloudFront | land-registry-api | HTTPS in front of EC2; all methods allowed, origin cache headers, HTTP→HTTPS redirect |

## 5.2 Storage and Database on AWS

Figures 5.2 and 5.3 show the two S3 buckets and the two DynamoDB tables created for the project. Every successful registration or transfer writes the deed to the private bucket and one item to LandDocuments with the S3 key, document hash, owner, action, transaction hash and (for MetaMask) the signer address. The blockchain stays the source of truth: if the DynamoDB write fails, the request still succeeds and returns a warning.

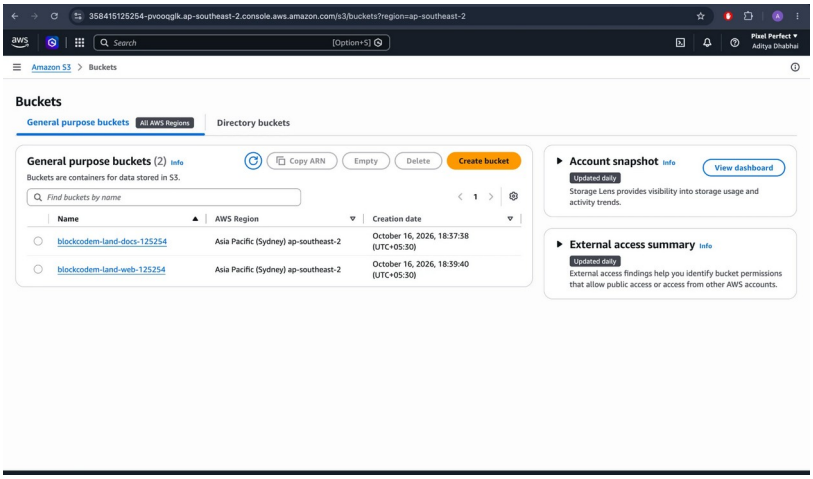



**Figure 5.2: Amazon S3: document bucket and website bucket**

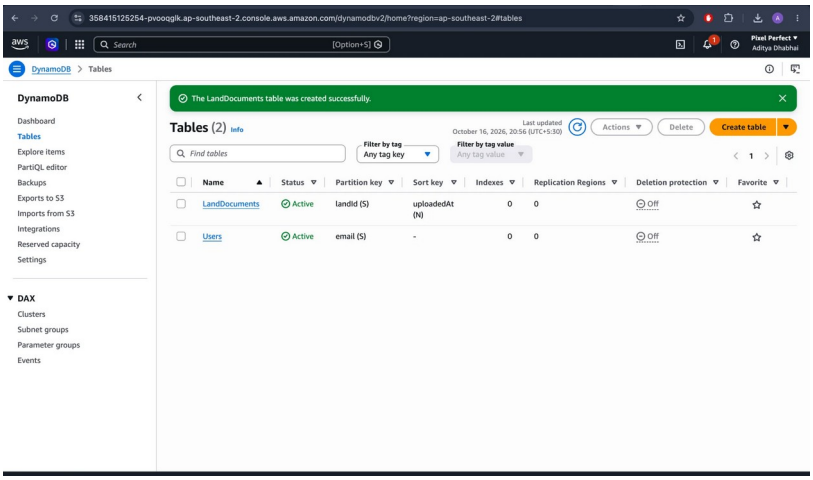



**Figure 5.3: Amazon DynamoDB: Users and LandDocuments tables**

## 5.3 Compute, IAM and HTTPS

The EC2 instance is launched with the IAM role of Figure 5.4, so the AWS SDK obtains temporary credentials automatically and no access key is stored anywhere. deploy/ec2-setup.sh installs the code, writes .env with a random wallet key and JWT secret, starts Ganache with a persistent database directory, deploys the contract once and starts the API under pm2 with auto-start on reboot. MetaMask accepts only HTTPS RPC URLs, so a CloudFront distribution (Figure 5.6) was placed in front of the instance; the health check https://d1077v0av0oht1.cloudfront.net/api/health returns {"success":true,"storage":"s3"}.

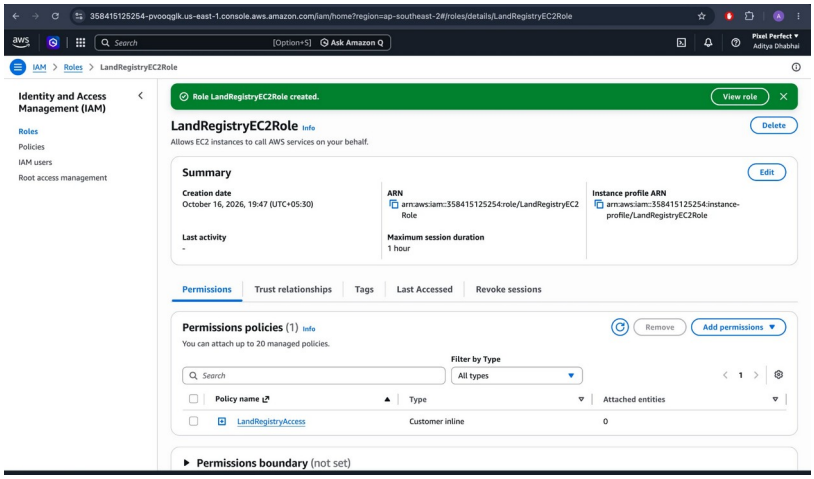



**Figure 5.4: IAM role LandRegistryEC2Role attached to the server**

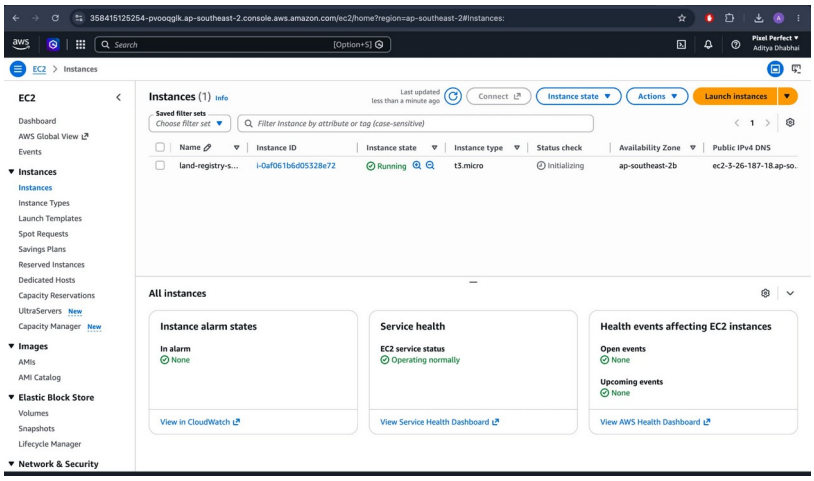



**Figure 5.5: Amazon EC2 instance land-registry-server running**

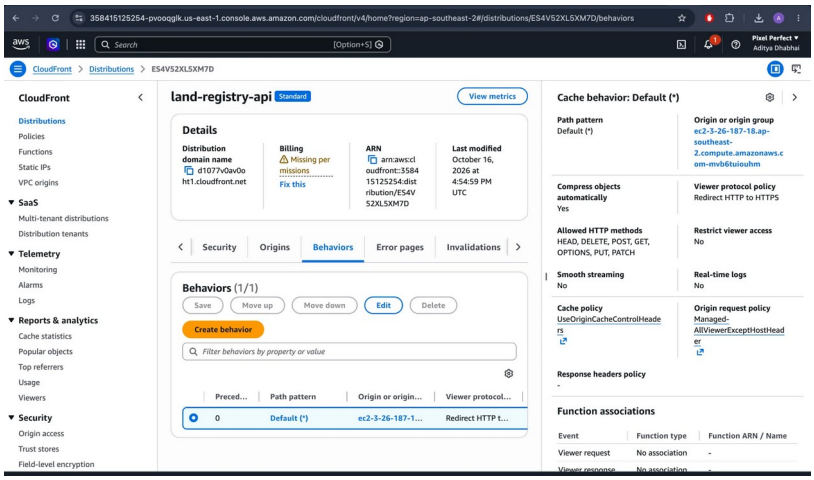



**Figure 5.6: Amazon CloudFront distribution and cache behaviour for the API**

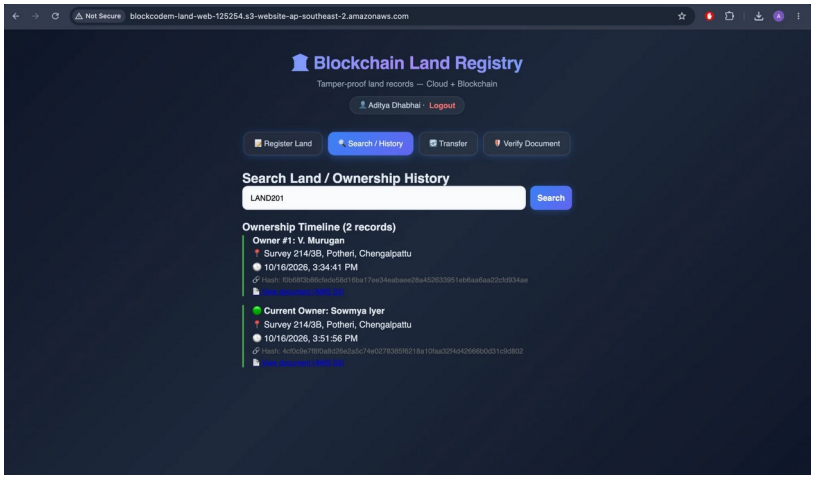



**Figure 5.7: Application served from the S3 website, with deed links from S3**

## 5.4 MetaMask Integration

MetaMask lets a registrar sign transactions with their own key instead of the server wallet. On Connect, the frontend reads /api/chain and adds the network "BlockCodeM Land Chain" (chain id 2602) to MetaMask with the CloudFront /api/rpc URL. This gateway forwards only read methods and eth_sendRawTransaction; eth_sendTransaction and eth_sign are blocked so nobody can use the server wallet through it. A logged-in registrar can request 5 test ETH for gas ("Get test ETH").

1. **Prepare:** the API checks the Land ID, hashes the PDF and uploads it to S3 (/api/land/prepare).
2. **Sign:** the browser calls registerLand() or transferOwnership() through MetaMask and the user confirms the transaction (Figure 5.8).
3. **Record:** the API reads the receipt, checks that it succeeded, was sent to the contract and that the on-chain hash and owner match, then indexes it in DynamoDB with the signer address (/api/land/record).

**Verify on blockchain:** the Verify tab can also compute the SHA-256 hash in the browser and call verifyHash() through MetaMask, so the document never leaves the user's computer (Figure 5.10).

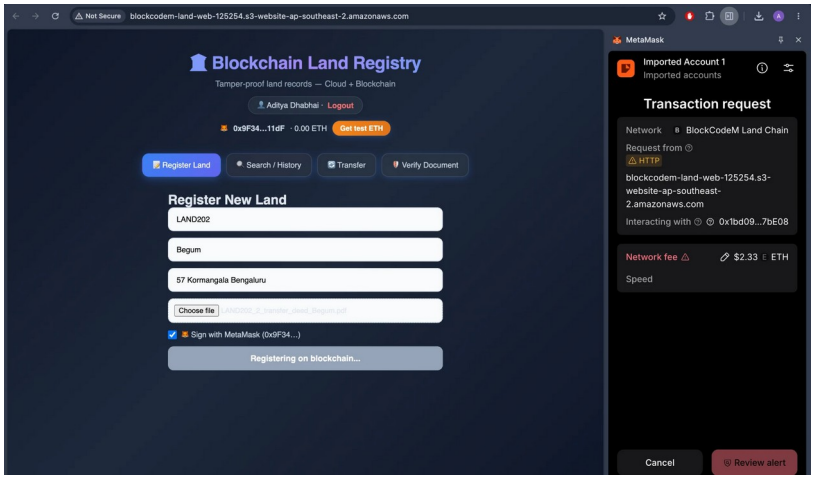



**Figure 5.8: MetaMask transaction request for registering LAND202**

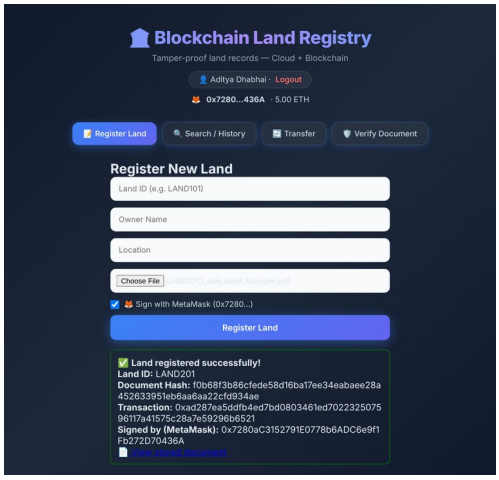



**Figure 5.9: Land registered with a MetaMask signature (signer address shown)**

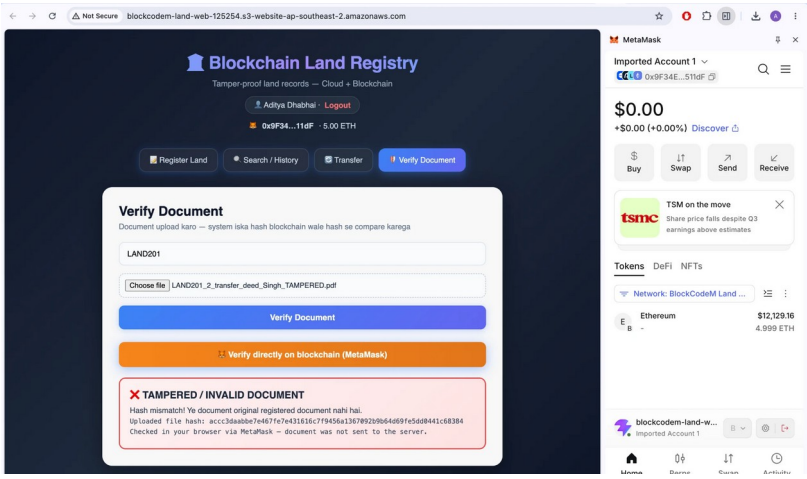



**Figure 5.10: Tampered deed detected directly on the blockchain through MetaMask**

# CHAPTER 6: TESTING AND VALIDATION

## 6.1 Testing Objectives

- Confirm that every functional requirement (register, transfer, history, verify, auth) works end-to-end on a real chain.
- Confirm that invalid input and unauthorised requests are rejected with clear messages.
- Confirm that tampered or outdated documents are always detected.

## 6.2 Types of Testing

- **Functional (UI) testing:** complete demo flow driven in the browser with Playwright (Chapter 6 screenshots).
- **API testing:** each endpoint called with curl / fetch using valid and invalid inputs.
- **Smart contract testing:** require() rules checked through duplicate and unknown Land IDs.
- **Cloud and wallet testing:** the same flow repeated on AWS through the S3 website and CloudFront, with and without MetaMask signing.
- **Performance testing:** 30 iterations of every operation, plus 50 concurrent history requests.

## 6.3 Test Cases

**Table 6.1: Functional and negative test cases**

| ID | Test case | Expected | Actual result | Status |
|---|---|---|---|---|
| TC01 | Signup with new email | Account created | "Signup successful! Please login." | Pass |
| TC02 | Signup with existing email | 400 error | 400 "Email already registered" | Pass |
| TC03 | Signup with missing fields | 400 error | 400 "All fields required" | Pass |
| TC04 | Login with wrong password | 401 error | 401 "Invalid email or password" | Pass |
| TC05 | Register without token | 401 error | 401 "Login required" | Pass |
| TC06 | Register with forged token | 401 error | 401 "Invalid or expired token" | Pass |
| TC07 | Register LAND101 with PDF | Tx mined, hash shown | docHash + txHash returned (Fig. 7.3) | Pass |
| TC08 | Register duplicate LAND101 | Rejected | 400 "already registered. Use Transfer" | Pass |
| TC09 | Upload .txt instead of PDF | Rejected | 400 "Only PDF files are allowed" | Pass |
| TC10 | Transfer LAND101 to new owner | History grows to 2 | 2 records shown (Fig. 7.5) | Pass |
| TC11 | Transfer unknown Land ID | Rejected | 400 "Land NOPE is not registered yet." | Pass |
| TC12 | Verify current deed | VERIFIED | verified: true (Fig. 7.6) | Pass |
| TC13 | Verify edited deed | TAMPERED | verified: false (Fig. 7.7) | Pass |
| TC14 | Verify old deed after transfer | Not valid | verified: false | Pass |
| TC15 | History of unknown Land ID | 404 error | 404 "Land NOPE is not registered" | Pass |
| TC16 | Register / verify without file | 400 error | 400 "Document (PDF) is required" | Pass |

| ID | Test case | Expected | Actual result | Status |
|---|---|---|---|---|
| TC17 | Two writes sent at the same time | Both mined | Both mined (write queue) | Pass |
| TC18 | Deed stored and listed on AWS | S3 link works | Pre-signed S3 link opens PDF (Fig. 5.7) | Pass |
| TC19 | Register / transfer signed in MetaMask | Signer on-chain | Signed by user address (Fig. 5.9) | Pass |
| TC20 | eth_sendTransaction sent to /api/rpc | Rejected | "Method not allowed" | Pass |
| TC21 | Verify tampered deed via MetaMask | TAMPERED | verified: false (Fig. 5.10) | Pass |

## 6.4 Validation of Inputs

- **Frontend:** every form checks that all fields and the file are present before calling the API.
- **File:** MIME type must be application/pdf; size at most 10 MB.
- **Business rules (contract):** a Land ID cannot be registered twice; transfer, history and verify require an existing Land ID.
- **Auth:** register and transfer require a valid, unexpired JWT.

## 6.5 Issues Found and Resolved

**Table 6.2: Issues found during testing and their fixes**

| Issue | Cause | Fix |
|---|---|---|
| Missing file gave HTTP 500 | hashFile() called without a file check | Fixed: 400 "Document (PDF) is required" |
| Unknown Land ID gave 500 | Contract revert passed through | Fixed: landExists() check, 404 |
| "Nonce too low" on parallel writes | Provider cached the pending nonce | Fixed: serialised queue, nonce from node |
| Transfer reverted (out of gas) on EC2 | Ganache returns an exact gas estimate | Fixed: 30% gas-limit buffer |
| Temp files left in uploads/ | Multer files never deleted | Fixed: unlink after each response |
| MetaMask refused the RPC URL | MetaMask needs an HTTPS RPC | Fixed: CloudFront HTTPS endpoint |

# CHAPTER 7: OUTPUT AND RESULTS

## 7.1 Application Screenshots

The following screenshots were captured from the running application (React frontend, Express backend, contract deployed on the local Ethereum node) during the complete demo flow for parcel LAND101. The same screens on AWS, and the MetaMask screens, are shown in Chapter 5.

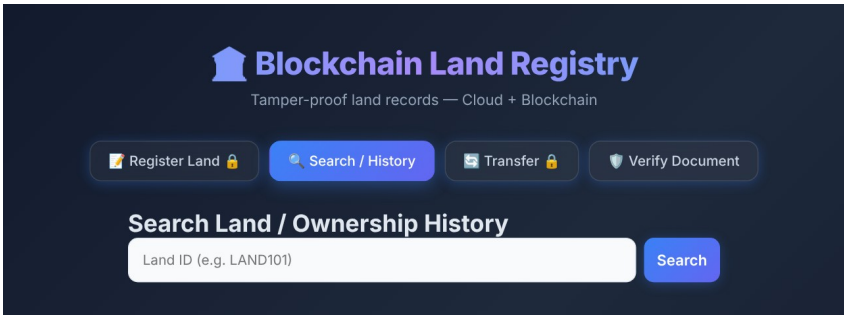



**Figure 7.1: Home page (public Search / History tab)**

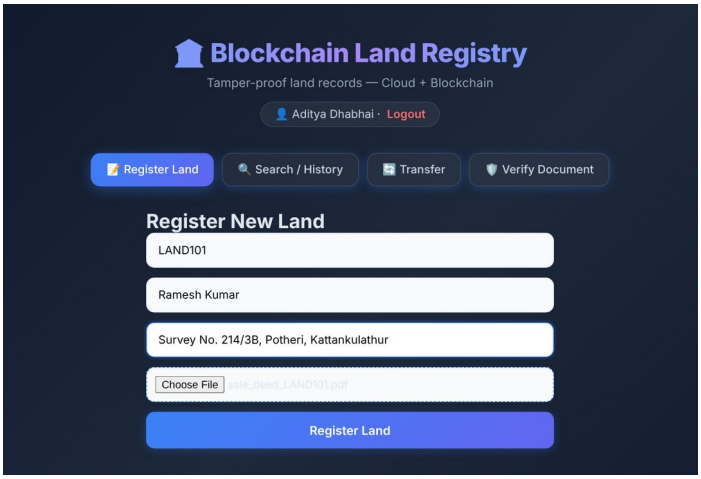



**Figure 7.2: Register New Land form with sale-deed PDF**

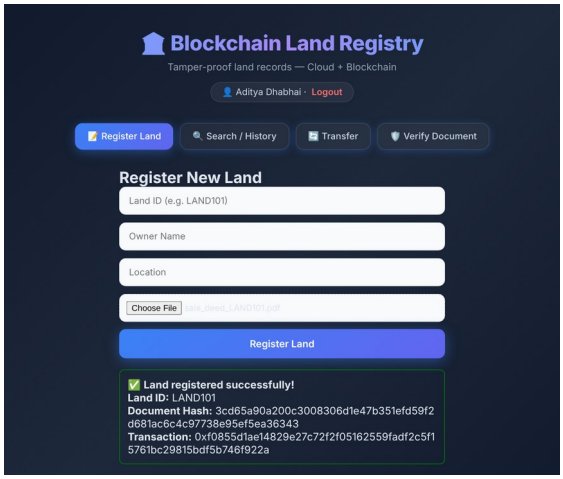



**Figure 7.3: Land registered on blockchain: document hash and transaction hash**

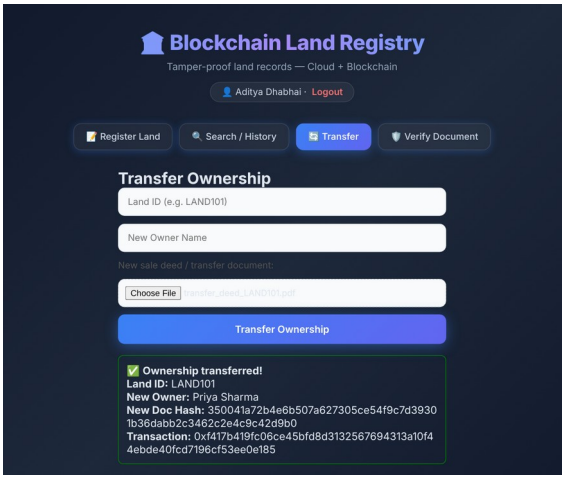



**Figure 7.4: Ownership transferred to Priya Sharma**

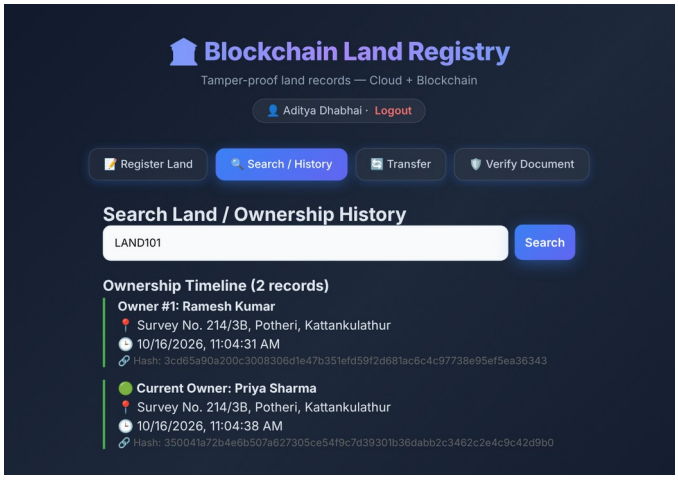



**Figure 7.5: Ownership timeline after transfer (2 records)**

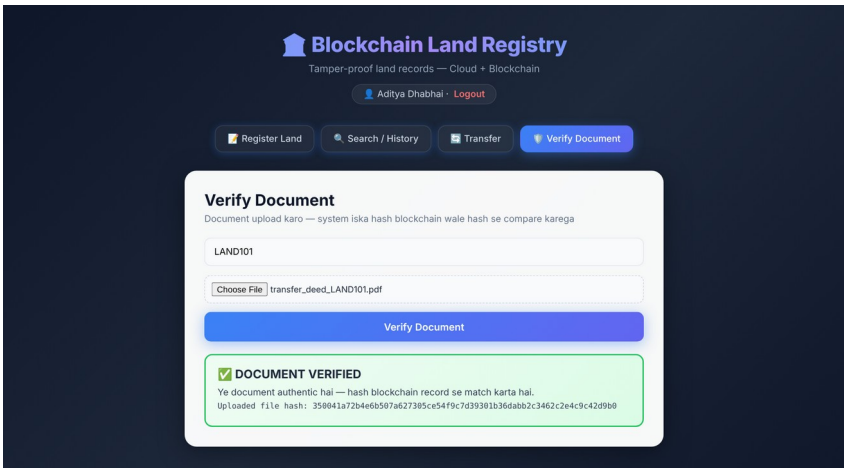



**Figure 7.6: Genuine document: DOCUMENT VERIFIED**

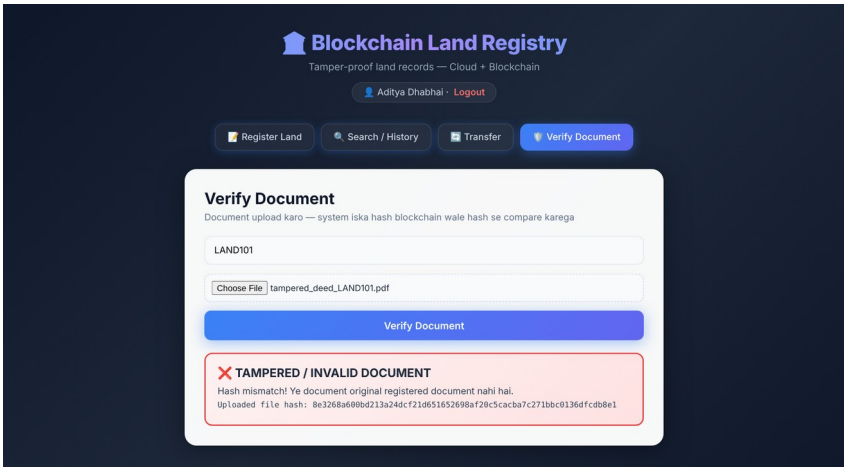



**Figure 7.7: Edited document: TAMPERED / INVALID DOCUMENT**

## 7.2 Output Explanation

- **Registration (Fig. 7.3):** the API returns the 64-hex SHA-256 hash of the deed and the Ethereum transaction hash, proving that the record was mined.
- **History (Fig. 7.5):** after the transfer the earlier owner, Ramesh Kumar, is still listed as Owner #1 with the original deed hash, and Priya Sharma is the current owner. Nothing is overwritten.
- **Verification (Fig. 7.6, 7.7):** the current transfer deed matches the chain. A copy whose buyer name and amount were edited produces a completely different hash and is flagged as tampered.

## 7.3 Performance Evaluation

Each operation was executed 30 times through the REST API on the local deployment (Hardhat Network with automine, Node.js 22). Write latency includes hashing, the contract transaction and waiting for the receipt. Cloud upload time is not included, because it depends on the network link to the cloud provider.

**Table 7.1: API response time (n = 30 per operation)**

| Operation | Avg (ms) | Min (ms) | Max (ms) | P95 (ms) |
|---|---|---|---|---|
| Register land (write tx) | 117.1 | 111.1 | 129.4 | 121.1 |
| Transfer ownership (write tx) | 114.4 | 107.8 | 123.4 | 122.4 |
| Get history (view call) | 15.2 | 13.7 | 18.3 | 17.1 |
| Verify genuine document | 15.7 | 13.4 | 20.3 | 19.1 |
| Verify tampered document | 15.4 | 14.1 | 17.1 | 16.3 |

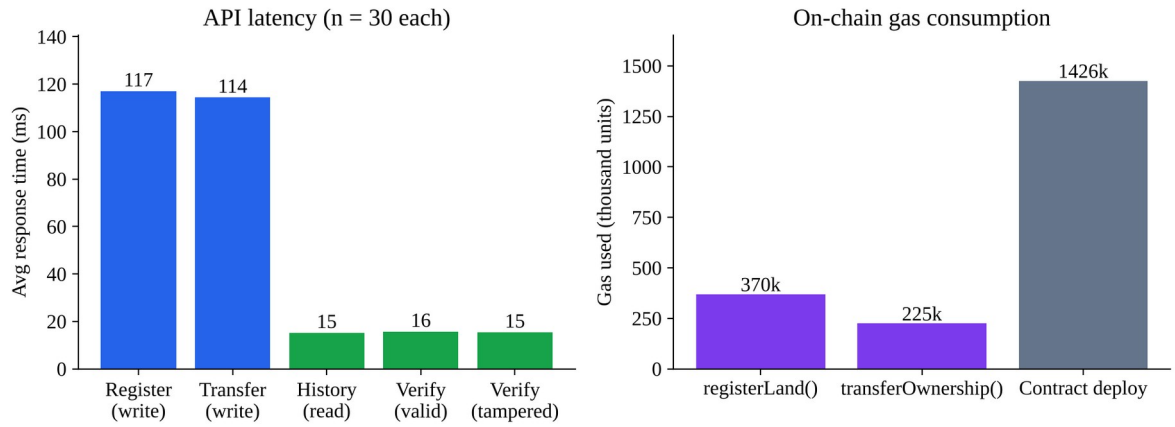



**Figure 7.8: API latency and on-chain gas consumption**

**Table 7.2: Blockchain cost and accuracy metrics**

| Metric | Measured value | Remark |
|---|---|---|
| Contract deployment gas | 1,425,762 | One-time cost |
| registerLand() gas | 369,571 (avg) | Grows with string length; LAND101 with long address used 458,781 |
| transferOwnership() gas | 224,566 (avg) | Cheaper: location reused |
| Tamper detection accuracy | 30/30 genuine verified, 30/30 tampered detected | 100% on test set |
| 50 concurrent history requests | 135.1 ms total | About 2.7 ms per request |
| Login (bcrypt compare + JWT) | 155.0 ms | bcrypt cost factor 10 |

**Observations:** read operations (history and verify) are about 7–8 times faster than writes because they are free eth_call requests that need no mining. Gas cost is dominated by string storage, so storing only a fixed-length hash instead of the document keeps every registration well under 0.5 million gas. On a public network the write latency would rise to the block time (about 12 seconds on Ethereum), while reads stay fast.

# CHAPTER 8: CONCLUSION AND FUTURE ENHANCEMENTS

## 8.1 Summary

The Blockchain Land Registry shows that a smart contract combined with cloud document storage gives a practical, tamper-evident land record system. Ownership records and their full history are immutable on the Ethereum ledger. Deeds are stored cheaply off-chain, and any party can check a document's authenticity in about 16 ms. The system is a complete web workflow (registrar authentication, registration, transfer, public search and public verification), deployed on AWS with S3, DynamoDB, EC2, IAM and CloudFront, and integrated with MetaMask for user-signed transactions.

## 8.2 Key Achievements

- Solidity contract with duplicate protection, append-only history, events and on-chain hash verification.
- Detection of every tampered document in testing (30/30), with no false positives on genuine deeds (30/30).
- Secure REST API with JWT, bcrypt, PDF-only uploads and server-side hashing.
- Full AWS deployment: S3 (deeds + website), DynamoDB (users + document index), EC2 (API + chain), IAM role (no keys in code) and CloudFront HTTPS.
- MetaMask integration: user-signed register / transfer, verified receipts, test-ETH faucet and in-browser on-chain verification.

## 8.3 Limitations

- The contract has no role check, so any funded wallet could write; the API controls who gets test ETH.
- The chain is a single Ganache node on one EC2 instance, not a public or multi-node network.
- The website itself is served over HTTP; only the API is behind CloudFront HTTPS.

## 8.4 Future Enhancements

- **Public testnet:** deploy the contract to Ethereum Sepolia so that anyone can audit it on a public explorer.
- **HTTPS website and custom domain:** put the S3 website behind CloudFront with an ACM certificate.
- **Monitoring:** CloudWatch logs, metrics and alarms for the API and the EC2 instance.
- **Role-based access:** Registrar, Owner and Verifier roles enforced in the contract (onlyRegistrar modifier).

# REFERENCES


[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum White Paper, 2014.

[3] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper, 2014.

[4] V. Thakur, M. N. Doja, Y. K. Dwivedi, T. Ahmad and G. Khadanga, "Land records on Blockchain for implementation of Land Titling in India," International Journal of Information Management, vol. 52, 101940, 2020.

[5] S. Krishnapriya and G. Sarath, "Securing Land Registration using Blockchain," Procedia Computer Science, vol. 171, pp. 1708–1715, 2020.

[6] M. Shuaib, S. M. Daud, S. Alam and W. Z. Khan, "Blockchain-based framework for secure and reliable land registry system," TELKOMNIKA, vol. 18, no. 5, pp. 2560–2571, 2020.

[7] Lantmäteriet, Telia, ChromaWay and Kairos Future, "The Land Registry in the Blockchain – Testbed," 2017.

[8] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

[9] J. Benet, "IPFS – Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

[10] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.

[11] Solidity Documentation, https://docs.soliditylang.org

[12] Hardhat Documentation, https://hardhat.org/docs

[13] ethers.js v6 Documentation, https://docs.ethers.org/v6/

[14] Cloudinary Upload API Reference, https://cloudinary.com/documentation/image_upload_api_reference

[15] Amazon S3 User Guide, https://docs.aws.amazon.com/s3/

[16] Amazon DynamoDB Developer Guide, https://docs.aws.amazon.com/dynamodb/

[17] Amazon CloudFront Developer Guide, https://docs.aws.amazon.com/cloudfront/

[18] MetaMask Developer Documentation, https://docs.metamask.io